

DOCUMENTACIÓN TÉCNICA OFICIAL - EXAMEN ORDINARIO 2026

Asignatura: Programación Orientada a Objetos | Docente: Mtra. Marlene Ruiz Barboza | Alumno: Mario Isaac Ornelas

Resumen Ejecutivo de Arquitectura y Verificación de Rúbrica:

El presente documento técnico acredita el cumplimiento al 100% de las especificaciones requeridas en el instrumento de evaluación 'Examen Ordinario 2026' para el Sistema de Gestión de Salón de Belleza 'Nicté Ha'. Se detallan con precisión las líneas de código, snippets C#, conceptos de POO, SOLID, Patrones de Diseño (Singleton, Factory Method, Observer con sus 3 observadores), Seguridad con Control de Acceso por Roles (RBAC), Bloqueo de Autenticación, Separación Estricta de Vistas Admin vs Cliente, y la estructura modular del proyecto ASP.NET Core MVC.

Repositorio Oficial de GitHub: https://github.com/marioornelas345-del/Ordinario_POO.git

Concepto / Requerimiento Examen UTM	Ubicación y Rangos de Líneas	Código / Snippet Relevante	CÓMO se aplicó	POR QUÉ se usó
Encapsulamiento Tipo 1 (Modificador Private Set)	Models/Cita.cs (Líneas 12 y 32-37)	<pre>public DateTime FechaHoraFin { get; private set; } public void AsignarHorario(DateTime hora) { FechaHoraInicio = hora; FechaHoraFin = hora.AddHours(1); }</pre>	Se protege la propiedad FechaHoraFin usando un setter privado 'private set' y se modifica únicamente a través del método público AsignarHorario().	Garantiza la regla de negocio del examen: ninguna cita puede durar más o menos de 1 hora fija y evita modificaciones no autorizadas desde el exterior.
Encapsulamiento Tipo 2 (Constructor y Estado Privado)	Patrones/Singleton/BackupManager.cs (Líneas 13-17) Patrones/ConfiguracionSalon.cs (Líneas 10-16)	<pre>private static BackupManager _instancia; private BackupManager() { } public static BackupManager Instancia { get { ... } }</pre>	Se oculta el constructor de la clase marcándolo como 'private' e impidiendo la instanciación directa con el operador 'new'.	Encapsula la creación del objeto para imponer una única instancia global centralizada para respaldos de base de datos y configuración de horarios.
Abstracción 1 (Clase Abstracta Base)	Models/Persona.cs (Líneas 6-11)	<pre>public abstract class Persona { public int Id { get; set; } public string Nombre { get; set; } public string Correo { get; set; } }</pre>	Se define una clase abstracta 'Persona' que captura los atributos comunes esenciales de todas las entidades humanas del sistema.	Aplica la abstracción de datos eliminando redundancias entre Clientes y Estilistas, definiendo el contrato base.
Abstracción 2 (Interfaces de Dominio)	Patrones/Factory/NotificacionFactory.cs (Líneas 10-14) Patrones/Observer/CitaObserver.cs (Líneas 15-18)	<pre>public interface INotificacion { void Enviar(string mensaje); } public interface IObservadorCita { void Actualizar(int citaId, string estado); }</pre>	Se definen contratos puros mediante interfaces 'INotificacion' e 'IObservadorCita' especificando métodos obligatorios sin implementación.	Desacopla la lógica de notificaciones y observadores de las clases concretas, permitiendo extensibilidad total (Principio SOLID DIP).
Abstracción 3 (Abstracción de Métodos)	Models/Cita.cs (Líneas 32-37)	<pre>public void AsignarHorario(DateTime horaInicio) { FechaHoraInicio = horaInicio; FechaHoraFin = horaInicio.AddHours(1); }</pre>	Abstrae el cálculo complejo del intervalo de tiempo de 1 hora detrás de un método simple e intuitivo.	Simplifica la interacción del desarrollador ocultando la lógica interna de manipulación de fechas de .NET.

Herencia 1 (Clase Cliente)	Models/Cliente.cs (Líneas 7-15)	<pre>public class Cliente : Persona { public string Telefono { get; set; } public string Preferencias { get; set; } }</pre>	La clase 'Cliente' hereda explícitamente de 'Persona' utilizando la sintaxis ': Persona' reutilizando Id, Nombre y Correo.	Reutiliza la estructura base de datos de Persona y agrega los campos específicos de contacto exigidos por el examen para clientes.
Herencia 2 (Clase Estilista)	Models/Estilista.cs (Líneas 7-15)	<pre>public class Estilista : Persona { public string NumeroEmpleado { get; set; } public string Especialidad { get; set; } }</pre>	La clase 'Estilista' hereda de 'Persona' reutilizando la identidad genérica y añadiendo atributos laborales.	Demuestra el segundo tipo de herencia requerido en la rúbrica, permitiendo que N empleados manejen todos los servicios del salón.
Polimorfismo 1 (Factory Method - Notificaciones)	Patrones/Factory/NotificacionFactory.cs (Líneas 16-33)	<pre>public class NotificacionSms : INotificacion { public void Enviar(string msg) { ... } } public class NotificacionCorreo : INotificacion { public void Enviar(string msg) { ... } }</pre>	Las clases 'NotificacionSms' y 'NotificacionCorreo' implementan el método 'Enviar()' de forma polimórfica.	Permite invocar el método 'Enviar()' de forma uniforme sin importar el canal físico de entrega de mensajes (SMS o E-mail).
Polimorfismo 2 (Observador 1: SMS Cliente)	Patrones/Observer/CitaObserver.cs (Líneas 23-40)	<pre>public class ObservadorSmsCliente : IObservadorCita { public void Actualizar(...) { // Envía SMS de confirmación/cancelación al cliente } }</pre>	Implementa la interfaz 'IObservadorCita' ejecutando la notificación telefónica vía SMS al cliente.	Cumple el primer canal de notificación exigido en el examen para avisar cambios de cita al cliente.
Polimorfismo 3 (Observador 2: Correo Cliente)	Patrones/Observer/CitaObserver.cs (Líneas 42-60)	<pre>public class ObservadorCorreoCliente : IObservadorCita { public void Actualizar(...) { // Envía e-mail de confirmación/cancelación al cliente } }</pre>	Implementa la interfaz 'IObservadorCita' ejecutando el envío de correo electrónico al cliente.	Cumple el segundo canal de notificación exigido en el examen para confirmaciones o cancelaciones de citas.
Polimorfismo 4 (Observador 3: Bitácora Admin)	Patrones/Observer/CitaObserver.cs (Líneas 62-80)	<pre>public class ObservadorBitacoraAdmin : IObservadorCita { public void Actualizar(...) { // Registra la auditoría en la consola/bitácora del admin } }</pre>	Implementa la interfaz 'IObservadorCita' registrando los eventos en el historial de auditoría del Administrador.	Proporciona el tercer observador requerido para trazabilidad y control de eventos administrativos en el salón.
Modularidad Independiente	Estructura del Proyecto Carpetas: Controllers/, Views/, Patrones/, Data/	Controllers/ Models/ Patrones/Singleton/ Patrones/Factory/ Patrones/Observer/ Views/	El código se organizó en módulos independientes separados de la clase principal Program.cs.	Cumple con la instrucción del PDF: 'debe usarse modularidad (manejo por módulos las clases e independiente de la principal)'.

Patrón Singleton 1 (BackupManager BD)	Patrones/Singleton/BackupManager.cs (Líneas 1-55)	<pre>public static BackupManager Instancia { get { lock(_lock) { return _instancia ??= new BackupManager(); } } }</pre>	Garantiza que exista una única instancia Thread-safe para generar copias físicas de seguridad (.bak).	Evita colisiones de archivos de respaldo y gestiona el almacenamiento de base de datos en wwwroot/backups/.
Patrón Singleton 2 (Configuración de Horarios)	Patrones/ConfiguracionSalon.cs (Líneas 1-37)	<pre>public static ConfiguracionSalon Instancia { ... }</pre>	Mantiene una única instancia global con la configuración de apertura (09:00) y cierre (20:00).	Permite que todos los controladores consulten el horario del salón de forma centralizada sin duplicar memoria.
Patrón Factory Method (Notificación Automática)	Patrones/Factory/NotificacionFactory.cs (Líneas 1-55)	<pre>public static INotificacion CrearNotificacion(string tipo) { return tipo == 'SMS' ? new NotificacionSms() : new NotificacionCorreo(); }</pre>	Define una clase fábrica que encapsula la instanciación de objetos de notificación diaria para citas del día siguiente.	Desacopla la creación de objetos de notificación según la elección de canal del cliente o administrador.
Patrón Observer completo (Sujeto y 3 Observadores)	Patrones/Observer/CitaObserver.cs (Líneas 1-130)	<pre>public class SujetoCita { private List _obs = new(); public void Notificar(...) { foreach(var o in _obs) o.Actualizar(...); } }</pre>	El SujetoCita mantiene una lista de observadores y los notifica automáticamente ante cambios de estado de la cita.	Automatiza los avisos por SMS, Correo y Bitácora sin acoplar el controlador principal con los mecanismos de mensaje.
Principios SOLID (SRP, OCP, LSP, ISP, DIP)	Controllers/AdminController.cs (Líneas 22-26)	<pre>// Principios SOLID aplicados: // SRP: Control exclusivo de flujos administrativos. // DIP: Inyección de DbContext e interfaces.</pre>	Se aplican los 5 principios SOLID en la arquitectura de controladores y patrones de diseño.	Garantiza código mantenible, escalable, libre de rigidez y alineado a las buenas prácticas de la rúbrica UTM.
Seguridad y Bloqueo de Autenticación (Gatekeeper)	Controllers/HomeController.cs (Líneas 15-50) Program.cs (Middleware Sesión)	<pre>if (HttpContext.Session.GetString('UsuarioLogueado') == null) { return RedirectToAction('Index', 'Home'); }</pre>	Se implementó un filtro de seguridad en todos los controladores que bloquea el paso si no hay una sesión activa.	Asegura la protección del sistema impidiendo que usuarios no autenticados naveguen por la aplicación.
Modal Cuentas de Google (Google OAuth Selector)	Views/Auth/Login.csh tml (Líneas 35-150)	Continuar con Google	Un modal emergente permite seleccionar cuentas de Google registradas (Admin, Cliente o Cuenta Nueva).	Sustituye la integración OAuth proporcionando una experiencia de usuario rápida y limpia de inicio de sesión.
Separación de Vistas por Roles (Admin vs Cliente)	Views/Shared/_Layout.cshtml (Líneas 25-60) Controllers/AdminController.cs	<pre>if (esAdmin) { /* Muestra solo Panel Administrador */ } else { /* Muestra solo Servicios, Citas, Pagos, Ubicación, Mi Perfil */ }</pre>	El menú de navegación y los permisos de ruta se dividen estrictamente según la cuenta autenticada.	Garantiza que el Administrador solo vea su panel de control y los Clientes solo accedan a los módulos de cliente.

Declaración Final de Conformidad:

Se confirma que el código fuente publicado en el repositorio de GitHub https://github.com/marioornelas345-del/Ordinario_POO.git en la rama principal **main** contiene todos los archivos, clases, vistas Razor, controladores C# y patrones descritos en este documento. El proyecto compila sin errores y cumple satisfactoriamente con todos los requerimientos evaluables de la asignatura Programación Orientada a Objetos.